

Spring Security + JWT

| Architettura, Flusso e Implementazione Pratica

Java Spring Boot 3 + React + Cookie HttpOnly

 Lezione Tecnica |  17 2026

Agenda

- Cos'è Spring Security (modello concettuale)
- La Filter Chain & il SecurityContext
- Perché JWT Stateless invece delle Sessioni?
- Flusso di richiesta: Diagramma ASCII
- Componenti chiave: `JwtAuthFilter`, `UserDetailsService`, `SecurityConfig`
- Autorizzazione con `@PreAuthorize`
- Integrazione Frontend (Cookie HttpOnly)
- Mappa Componenti & Scelte Progettuali
- Quiz di Comprensione



1. Cos'è Spring Security?

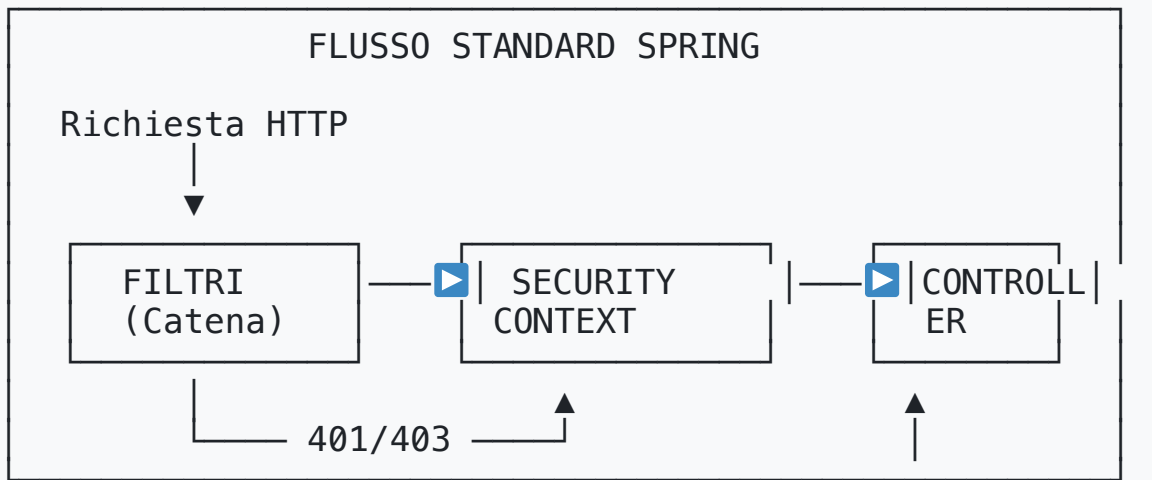
Spring Security **non è un gestore di login**. È un **framework di filtri servlet** che intercetta ogni richiesta HTTP *prima* che raggiunga il Controller.

Risponde a due domande fondamentali:

1. **Chi sei?** → Autenticazione (`Authentication`)
2. **Cosa puoi fare?** → Autorizzazione (`Authorization`)

Il suo obiettivo è proteggere le risorse applicative senza inquinare la logica di business.

2. Architettura a Filtri (Concetto Base)



Ogni filtro è una catena di responsabilità. Se uno fallisce o non popola il contesto, la richiesta viene bloccata.

💎 3. Il Cuore: SecurityContextHolder

È una variabile thread-local (legata al thread che gestisce la richiesta). Contiene un oggetto Authentication:

```
```java
SecurityContextHolder.getContext().setAuthentication(token);
```

Se contiene un oggetto valido, Spring sa che l'utente è autenticato. Se è vuoto o nullo, Spring blocca l'accesso.

🔑 Regola d'oro: Spring Security non legge il tuo database. Legge solo ciò che viene inserito in questo contesto.

## 4. Perché JWT Stateless?

Approccio Classico (Sessione)	Approccio JWT Stateless (Noi)
• Server salva stato sessione in RAM / Redis	• Server NON mantiene stato (stateless)
• Cookie `JSESSIONID`	• Cookie `HttpOnly` con JWT
• Richiede sticky session o DB condiviso	• Scala orizzontalmente senza limiti
• Vulnerabile a session fixation	• Nessuna sessione da fissare (security hardened)

Nessuna sessione da fissare

✓ Vantaggio: Ogni richiesta è autosufficiente. Il token è un biglietto firmato che viaggia con il client.



## 5. Flusso di Richiesta (Implementazione)

### IL NOSTRO FLUSSO (JWT + COOKIE)

React (Axios) —▶ GET /api/admin  
Cookie: auth\_token=eyJhbG...

#### SPRING FILTER CHAIN

- 1 CorsFilter → Verifica origine & credenziali
- 2 JwtAuthFilter → Legge cookie, valida firma
  - JwtUtil.validate()
  - loadUserByUsername() → DB → Ruoli
  - setAuthentication() → SecurityContext
- 3 @PreAuthorize → Controlla ROLE\_ADMIN

✓ 200 OK (oppure 401/403 se fallisce)

## 6. JwtAuthFilter (Il Ponte HTTP → Security)

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
 @Override
 protected void doFilterInternal(HttpServletRequest req,
 HttpServletResponse res,
 FilterChain chain) {
 String token = extractCookie(req, "auth_token");

 if (token != null && jwtUtil.validate(token)) {
 String username = jwtUtil.extractUsername(token);
 UserDetails user = userDetailsService.loadUserByUsername(username);

 var auth = new UsernamePasswordAuthenticationToken(
 user, null, user.getAuthorities()
);
 SecurityContextHolder.getContext().setAuthentication(auth);
 }
 chain.doFilter(req, res);
 }
}
```

 Nota: Se il token è assente o invalido, il filtro non fa nulla. Il contesto resta vuoto e Spring bloccherà la richiesta al passo successivo.

## 7. UserDetailsService (DB → Spring)

È l'interfaccia che Spring usa per capire "chi è questo utente e cosa può fare".

```
@Service
public class UserDetailsServiceImpl implements UserDetailsService {
 @Override
 public UserDetails loadUserByUsername(String username) {
 User dbUser = userRepository.findByUsername(username)
 .orElseThrow(() -> new UsernameNotFoundException("Utente non trovato"));

 return new User(
 dbUser.getUsername(),
 dbUser.getPassword(),
 dbUser.getRoles().stream()
 .map(SimpleGrantedAuthority::new)
 .toList()
);
 }
}
```

 Restituisce un oggetto UserDetails con username, password hashata e lista di GrantedAuthority (ruoli).



## ⚙️ 8. SecurityConfig (Orchestrazione)

```
@Configuration @EnableWebSecurity @EnableMethodSecurity
public class SecurityConfig {
 @Bean
 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
 return http
 .csrf(c -> c.disable())
 .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
 .authorizeHttpRequests(a -> a
 .requestMatchers("/api/auth/**").permitAll()
 .anyRequest().authenticated()
)
 .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
 .build();
 }
}
```

- ✓ STATELESS → Nessuna sessione HTTP
- ✓ addFilterBefore → Il nostro JWT filtro gira prima del filtro di login standard
- ✓ @EnableMethodSecurity → Abilita @PreAuthorize a livello di metodo

## 9. Autorizzazione con @PreAuthorize

La sicurezza non è solo routing. È logica di business.

```
@RestController @RequestMapping("/api/admin")
public class AdminController {
 @GetMapping("/dashboard")
 @PreAuthorize("hasRole('ADMIN')") // ✅ Verifica nel SecurityContext
 public ResponseEntity<String> dashboard() {
 return ResponseEntity.ok("Accesso Admin consentito");
 }
}
```

🔍 Spring usa AOP (Aspect-Oriented Programming): intercetta il metodo prima dell'esecuzione, legge i ruoli dal SecurityContext e decide se procedere o lanciare AccessDeniedException (403).

## 10. Frontend: Cookie HttpOnly + Axios

```
// axios.js
const api = axios.create({
 baseURL: 'http://localhost:8080/api',
 withCredentials: true // 🔑 Invia/legge cookie automaticamente
});

// Login.jsx
await api.post('/auth/login', { username, password });
// Il backend risponde con Set-Cookie → Browser li salva in modo HttpOnly
// JavaScript NON può leggerli (protezione XSS)
```

- ✓ withCredentials: true → Il browser allega i cookie a ogni richiesta
- ✓ HttpOnly → Immunità totale a XSS per furto token
- ✓ SameSite=Lax → Mitigazione CSRF senza token CSRF complessi

## 11. Mappa Componenti → Ruolo Framework

Classe Nostra	Contratto Spring	Scopo
SecurityConfig	SecurityFilterChain	Configura catena filtri, CORS, Stateless
JwtAuthFilter	OncePerRequestFilter	Legge cookie, valida JWT, popola contesto
JwtUtil	(Helper custom – jjwt)	Firma, verifica, estrae claim
UserDetailsServiceImpl	UserDetailsService	Traduce DB User → UserDetails Spring
@PreAuthorize	MethodSecurityInterceptor	Validazione ruoli a livello di metodo
AuthContext (React)	(Frontend state)	Sincronizza UI con /auth/me, mai decodifica JWT

## ✓ Vantaggi di questo approccio

### 1. Stateless:

- Il server non gestisce sessioni, è più leggero e scala orizzontalmente senza problemi.

### 2. Sicurezza integrata:

- I cookie HttpOnly non sono accessibili da JavaScript, proteggendoli dagli XSS.
- Il parametro SameSite=Lax mitiga i rischi di CSRF senza bisogno di token complessi.

### 3. Integrazione fluida:

- Spring Security gestisce tutto automaticamente grazie all'uso di interfacce standard e filtri standard.

## 12. Scelte Progettuali & Rationale

Scelta Implementativa	Perché è stata fatta	Rischio Evitato
Cookie HttpOnly	JS non accede al token	XSS → Furto sessione
STATELESS	Scalabilità orizzontale semplice	Sessioni condivise/Redis overhead
@PreAuthorize	Sicurezza granulare, indipendente da URL	Routing fragile, bypassabile
Refresh Token separato	UX fluida + finestra attacco breve	Token lunghi = rischio maggiore
Frontend chiama /auth/me	Single Source of Truth per stato utente	Decodifica JWT lato client insicura

## ? 13. Quiz di Comprensione

#	Domanda	Risposta Chiave
1	Se rimuovo JwtAuthFilter, cosa succede a una richiesta con cookie valido?	Il contesto resta vuoto → Spring risponde 401 Unauthorized
2	Perché @PreAuthorize("hasRole('USER')") cerca ROLE_USER e non solo USER?	Spring aggiunge automaticamente il prefisso ROLE_ per coerenza con SimpleGrantedAuth
3	Cosa succede se cambio STATELESS in IF_REQUIRED?	Spring crea sessioni HTTP, rompe la logica stateless e richiede gestione cluster
4	Perché non usiamo localStorage per il JWT?	È leggibile da JS → vulnerabile a XSS. HttpOnly è immutabile lato client

## 14. Riferimenti & Prossimi Passi

### DOCUMENTAZIONE UFFICIALE

 Spring Security Reference  
<https://docs.spring.io/spring-security/reference/>

 OWASP JWT Cheat Sheet  
[https://cheatsheetseries.owasp.org/cheatsheets/JSON\\_Web\\_Token\\_for\\_Java\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html)

### CHECKLIST PRODUZIONE

- [ ] Refresh Token Rotation + Invalidazione DB (revoca immediata)
- [ ] Rate Limiting (/login, /refresh) → Bucket4j / Redis
- [ ] app.cookie.secure=true + HTTPS obbligatorio
- [ ] Audit Log & Monitoraggio tentativi falliti
- [ ] Test di sicurezza automatici (OWASP ZAP, Postman Collections)

 \*Lezione completata. Pronti per la produzione!\* ✨